SIDDHARTH INSTITUTE OF ENGINEERING & TECHNOLOGY
(AUTONOMOUS)

(Approved by AICTE, New Delhi & Affiliated to JNTUA, Ananthapuramu)
(Accredited by NBA for EEE, Mech., ECE & CSE and Accredited by NAAC with A+ Grade)
Siddartha Nagar, Narayanavanam Road, Puttur-517583

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

LAB MANUAL

DEPARTMENT OF COMPUTER SCIENCE AND ENGINNEERING

(23CS0521)COMPUTER NETWORKS & INTERNET PROTOCOLS LAB
(Common to CSE&CCC)

III B TECH – I SEMESTER

Academic Year : _____
Name : _____
Roll Number : _____
Year & Branch : _____
Section : _____

SIDDHARTH INSTITUTE OF ENGINEERING & TECHNOLOGY

Vision

To become an eminent academic institute in education and research, producing global leaders in science and technology for the betterment of mankind.

Mission

M1. To provide broad-based education and contemporary knowledge by adopting modern teaching-learning methods.
M2. To inculcate a spirit of research and innovation in students through industrial interactions.
M3. To develop individuals' potential to the fullest extent so that they can emerge as gifted leaders in their fields.

Department of Computer Science and Engineering

Vision

To become a reputed department of Computer Science and Engineering, producing competent professionals with strong research and innovation skills, while inculcating moral values and societal concerns.

Mission of the Department

M1. To educate students to become highly qualified computer engineers with strong commitment to professional ethics.
M2. To inculcate innovative and research-oriented thinking in computer science and related interdisciplinary areas, enabling students to provide advanced professional services to society.
M3. To prepare students with industry-ready knowledge and entrepreneurial skills by introducing industry-oriented educational programs.

Program Educational Objectives (PEOs)

PEO1. Graduates will possess fundamental and advanced knowledge in science, mathematics, computer science, and allied engineering disciplines, enabling them to analyse, design, and develop solutions for real-world problems.
PEO2. Graduates will serve in industry, consultancy, and government organizations, or pursue higher education and research.
PEO3. Graduates will demonstrate professional leadership, effective communication, teamwork, ethical values, and lifelong learning abilities.

Programme Outcomes (POs)

PO1 – Engineering Knowledge : Apply the knowledge of mathematics, science, engineering fundamentals, and an engineering specialization to the solution of complex engineering problems.
PO2 – Problem Analysis : Identify, formulate, review research literature, and analyse complex engineering problems to reach substantiated conclusions using first principles of mathematics, natural sciences, and engineering sciences.
PO3 – Design/Development of Solutions : Design solutions for complex engineering problems, and design system components or processes that meet specified needs with appropriate consideration for public health and safety, and cultural, societal, and environmental aspects.
PO4 – Conduct Investigations of Complex Problems : Use research-based knowledge and methods including the design of experiments, analysis and interpretation of data, and synthesis of information to provide valid conclusions.
PO5 – Modern Tool Usage : Create, select, and apply appropriate techniques, resources, and modern engineering and IT tools, including prediction and modelling, to complex engineering activities with an understanding of their limitations.
PO6 – The Engineer and Society : Apply contextual knowledge to assess societal, health, safety, legal, and cultural issues, and understand the consequent responsibilities relevant to professional engineering practice.
PO7 – Environment and Sustainability : Understand the impact of professional engineering solutions in societal and environmental contexts, and demonstrate knowledge of, and the need for, sustainable development.
PO8 – Ethics : Apply ethical principles and commit to professional ethics, responsibilities, and norms of engineering practice.
PO9 – Individual and Team Work : Function effectively as an individual, and as a member or leader in diverse teams, and in multidisciplinary settings.
PO10 – Communication : Communicate effectively on complex engineering activities with the engineering community and society at large. This includes the ability to comprehend and write effective reports, design documentation, make effective presentations, and give and receive clear instructions.
PO11 – Project Management and Finance : Demonstrate knowledge and understanding of engineering and management principles, and apply these to one's own work as a member and leader in a team, to manage projects in multidisciplinary environments.
PO12 – Life-long Learning : Recognize the need for, and possess the ability to engage in, independent and life-long learning in the broad context of technological change.

Programme Specific Outcomes (PSOs)

PSO1 – Architecture of Computer Systems : Ability to visualize and articulate computer hardware and software systems for various complex applications.

PSO2 – Design and Development of Computer Programs : Ability to design and develop computer-based systems in domains such as algorithms, networking, web design, cloud computing, IoT, data analytics, and mobile applications of varying complexity.

PSO3 – Applications of Computing and Research Ability : Ability to apply computing knowledge in various domains to identify research gaps and provide innovative solutions to new ideas and challenges.

SIDDHARTH INSTITUTE OF ENGINEERING & TECHNOLOGY

Computer Laboratory – Do's and Don'ts

Do's

1. Follow the correct procedures for starting and shutting down the computer to avoid data loss and damage to components.
2. Use only the specific program or site assigned by your lab faculty.
3. Report any problems or damages immediately to the Lab In-charge.
4. Use a light touch on the keyboard while typing.
5. Save your work frequently while working on documents.
6. Log out after finishing your session on any lab computer.
7. Shut down the computer properly before leaving the lab.
8. Know the location of the fire extinguisher and first aid box, and learn how to use them in case of an emergency.
9. Read and understand the experiment procedure thoroughly before coming to the laboratory.

Don'ts

1. Do not move or reconfigure computers and peripherals without approval from the Lab In-charge.
2. Students must not install software on lab computers. Contact the Lab In-charge for any specific software requirements.
3. Do not bang on the keyboard keys.
4. Do not open the metallic covers of computers or peripheral devices, especially when the power is on.
5. Never open email attachments from unknown sources.
6. Keep all liquids away from computers and equipment to prevent spills, electrical shocks, or malfunction.
7. Do not insert metal objects such as clips, pins, or needles into computer casings, as they may cause fire hazards.
8. Do not touch, connect, or disconnect any plug or cable without the Lab In-charge's permission.

S.No	Date	Name of the Experiment	Page No.	Sign.
01		Data link layer framing methods		
02		CRC code for the polynomials		
03		Dijkstra's algorithm		
04		Subnet of hosts and obtain a broadcast tree for the subnet.		
05		Distance vector routing algorithm		
06		Encryption and data decryption.		
07		Leaky bucket algorithm.		
08		Frame sorting technique used in buffers.		
09		Programs using Wireshark		
10		Nmap scan.		
11		Operating System Detection using Nmap.		
12		NS2 Simulator		

INDEX

III B.Tech–I Sem.

L	T	P	C

(23CS0521) COMPUTER NETWORKS & INTERNET PROTOCOL LAB

(Common to CSE& CCC)

COURSE OBJECTIVES

The course is designed to
1. To understand the working principle of various communication protocols.
2. To understand the network simulator environment and
3. To visualize a network topology observe its performance
4. To analyze the traffic flow and the contents of protocol frames.
5. Familiarize with the applications of Internet.

COURSE OUTCOMES

After completion of the course, students will be able to
1. Design scripts for Wired network simulation
2. Design scripts of static and mobile wireless networks simulation
3. Analyze the data traffic using tools
4. Design JAVA programs for client-server communication
5. Construct a wired and wireless network using the real hardware
6. Implement basic network security mechanisms and simulate secure data transmission scenarios.

LIST OF EXPERIMENTS

1. Implement the data link layer framing methods such as character stuffing and bit stuffing.
2. Write a program to compute CRC code for the polynomials CRC-12, CRC-16 and CRC CCIP. Develop a simple data link layer that performs the flow control using the sliding window protocol, and loss recovery using the Go-Back-N mechanism.
3. Implement Dijkstra's algorithm to compute the shortest path through a network.
4. Take an example subnet of hosts and obtain a broadcast tree for the subnet.
5. Implement distance vector routing algorithm for obtaining routing tables at each node.
6. Implement data encryption and data decryption.
7. Write a program for congestion control using Leaky bucket algorithm.
8. Write a program for frame sorting technique used in buffers.
9. Programs using Wireshark:
   i. Starting Wireshark
   ii. Packet Capture Using Wireshark
   iii. Viewing Captured Traffic
   iv. Analysis and Statistics & Filters
10. How to run Nmap scan.
11. Operating System Detection using Nmap.
12. Do the following using NS2 Simulator:
    i. NS2 Simulator – Introduction
    ii. Simulate to Find the Number of Packets Dropped
    iii. Simulate to Find the Number of Packets Dropped by TCP/UDP
    iv. Simulate to Find the Number of Packets Dropped due to Congestion
    v. Simulate to Compare Data Rate & Throughput
    vi. Simulate to Plot Congestion for Different Source/Destination
    vii. Simulate to Determine the Performance with respect to transmission of Packets

EXP – 01	Implement the data link layer framing methods such as character stuffing and bit stuffing.
Date:	

Aim:
To implement **Character Stuffing** and **Bit Stuffing** techniques for reliable data transmission at the Data Link Layer.

Description:
In data communication, the **Data Link Layer** ensures error-free and clear transmission of data frames.

Character Stuffing: Uses special characters like STX (start), ETX (end). If such characters appear in the data, a dummy character (DLE) is inserted before them.

Bit Stuffing: Uses a flag pattern 01111110. If five consecutive 1's appears in the data, a 0 is inserted to avoid confusion with the flag.

Both techniques help the receiver correctly identify frame boundaries.

Program 1: Character Stuffing (Java Program)

```
import java.util.Scanner;

public class CharacterStuffing {
   public static void main(String[] args) {
      Scanner sc = new Scanner(System.in);

      System.out.print("Enter the data: ");
      String data = sc.next();

      StringBuilder stuffed = new StringBuilder();

      // Add STX
      stuffed.append("STX");

      // Stuffing process
      for (int i = 0; i < data.length(); i++) {
         char ch = data.charAt(i);
         if (ch == 'E') {   // Treat 'E' as special char
            stuffed.append('D'); // Insert DLE
        }
         stuffed.append(ch);
      }

      // Add ETX
      stuffed.append("ETX");
      System.out.println("Stuffed Frame: " + stuffed.toString());

      sc.close();
   }
}
```

Output:

Enter the data: ABCDE
Stuffed Frame:

Program 2: Bit Stuffing (Java Program)

```
import java.util.Scanner;

public class BitStuffing {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter binary data: ");
        String data = sc.next();
        StringBuilder stuffed = new StringBuilder();

        // Start flag
        stuffed.append("01111110");
        int count = 0;
        for (int i = 0; i < data.length(); i++) {
            char bit = data.charAt(i);
            stuffed.append(bit);

            if (bit == '1') {
                count++;
                if (count == 5) {
                    stuffed.append('0'); // Stuff 0
                    count = 0;
                }
            } else {
                count = 0;
            }
        }

        // End flag
        stuffed.append("01111110");
        System.out.println("Bit Stuffed Frame: " + stuffed.toString());
        sc.close();
    }
}
```

Output:

Enter binary data: 0111111110
Bit Stuffed Frame:

Possible Viva Questions:

1. What is framing in the Data Link Layer?
2. Explain the difference between **Character Stuffing** and **Bit Stuffing** .
3. What is the purpose of the flag 01111110 in Bit Stuffing?
4. At which OSI layer are stuffing techniques used?
5. Why do we need stuffing techniques in communication?

Result:

EXP – 02	**Write a program to compute CRC code for the polynomials CRC-12, CRC-16 and CRC CCIP. Develop a simple data link layer that performs the flow control using the sliding window protocol, and loss recovery using the Go-Back-N mechanism.**
Date:	

Aim:

To implement

1. **CRC code generation** using given polynomials (CRC-12, CRC-16, CRC-CCITT),
2. **Sliding Window Protocol** for flow control, and
3. **Go-Back-N ARQ** for loss recovery.

Description:

CRC (Cyclic Redundancy Check):

CRC is an error-detecting code used in digital networks. The sender appends a CRC value (remainder after polynomial division) to the data, and the receiver checks if the remainder is zero.

Sliding Window Protocol:

It is a flow control protocol where multiple frames are sent before waiting for an acknowledgment. It uses a window of size N to keep track of sent frames.

Go-Back-N ARQ:

It is an error-control method where, if one frame is lost, all subsequent frames are retransmitted from that frame onwards.

Program 1: CRC Code (Java Program)

```
import java.util.Scanner;

public class CRC {
   public static void main(String[] args) {
      Scanner sc = new Scanner(System.in);
      System.out.print("Enter data: ");
      String data = sc.next();
      System.out.print("Enter divisor: ");
      String key = sc.next();
      int dlen = data.length(), klen = key.length();

      // Append (klen-1) zeros
      StringBuilder temp = new StringBuilder(data);
      for (int i = 0; i < klen - 1; i++) temp.append("0");

      char[] t = temp.toString().toCharArray();
      char[] k = key.toCharArray();
      // Division (XOR)
      for (int i = 0; i < dlen; i++) {   // FIX: run only till dlen-1
         if (t[i] == '1') {
            for (int j = 0; j < klen; j++) {
               t[i + j] = (t[i + j] == k[j]) ? '0' : '1';
           }
        }
      }
      // Remainder = CRC
      String crc = new String(t, dlen, klen - 1);
      System.out.println("CRC: " + crc);
      sc.close();
```

```
    }
}
```

Output:

```
Enter data: 110100111
Enter divisor: 1011
CRC:
```

Program 2: Sliding Window Protocol (Java Program)

```
import java.util.Scanner;

public class SlidingWindow {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        System.out.print("Enter number of frames to send: ");
        int n = sc.nextInt();

        System.out.print("Enter window size: ");
        int windowSize = sc.nextInt();

        int[] frames = new int[n + 1];
        for (int i = 1; i <= n; i++)
            frames[i] = i;

        System.out.println("\nTransmitting frames...");
        for (int i = 1; i <= n; i += windowSize) {
            System.out.print("Window: ");
            for (int j = i; j < i + windowSize && j <= n; j++) {
                System.out.print(frames[j] + " ");
            }
            System.out.println("\nAcknowledgment received for all above frames");
        }

        sc.close();
    }
}
```

Output:

```
Enter number of frames to send: 8
Enter window size: 3
```

Program 3: Go-Back-N ARQ (Java Program)

```
import java.util.Scanner;

public class GoBackNARQ {
   public static void main(String[] args) {
      Scanner sc = new Scanner(System.in);

      System.out.print("Enter window size: ");
      int n = sc.nextInt();

      System.out.print("Enter number of frames to transmit: ");
      int nf = sc.nextInt();

      int i = 1;
      while (i <= nf) {
         int k = 0;
         while (k < n && i <= nf) {
            System.out.println("Sent Frame " + i);
            i++;
            k++;
         }
         System.out.println("Acknowledgment for all above frames received\n");
      }

      sc.close();
   }
}
```

Output:

```
Enter window size: 4
Enter number of frames to transmit: 10
```

Possible Viva Questions:

1. What is the difference between **Error Detection** and **Error Correction** ?
2. How does **CRC** detect errors in a message?
3. What is the main advantage of the **Sliding Window Protocol** ?
4. Differentiate between **Go-Back-N** and **Stop-and-Wait ARQ** .
5. What happens if an acknowledgment is lost in Go-Back-N ARQ?

Result:

EXP – 03	Implement Dijkstra's algorithm to compute the shortest path through a network.
Date:	

Aim:
To implement Dijkstra's algorithm and find the shortest path from a source node to all other nodes in a network.

Description:
The program represents the network using an **adjacency matrix**.
It starts from the **source vertex (0)** .
Using **Dijkstra's algorithm** , it repeatedly selects the unvisited vertex with the smallest distance and updates the distances of its neighboring vertices.
Finally, it prints the shortest distance from the source to every vertex in the network.

Program: Dijkstra's Algorithm (Java Program)

```
import java.util.*;

public class DijkstraAlgorithm {

   static final int V = 6;

   // Function to find the vertex with the minimum distance
   int minDistance(int dist[], boolean visited[]) {
      int min = Integer.MAX_VALUE;
      int minIndex = -1;

      for (int v = 0; v < V; v++) {
         if (!visited[v] && dist[v] < min) {
            min = dist[v];
            minIndex = v;
        }
      }
      return minIndex;
   }

   // Dijkstra's Algorithm
   void dijkstra(int graph[][], int src) {
      int dist[] = new int[V];
      boolean visited[] = new boolean[V];

      Arrays.fill(dist, Integer.MAX_VALUE);
      dist[src] = 0;

      for (int count = 0; count < V - 1; count++) {
         int u = minDistance(dist, visited);
         visited[u] = true;

         for (int v = 0; v < V; v++) {
            if (!visited[v] &&
               graph[u][v] != 0 &&
```

```
                dist[u] != Integer.MAX_VALUE &&
                dist[u] + graph[u][v] < dist[v]) {

                dist[v] = dist[u] + graph[u][v];
            }
          }
        }

        // Print shortest distances
        System.out.println("Vertex\tDistance from Source");
        for (int i = 0; i < V; i++) {
            System.out.println(i + "\t\t" + dist[i]);
        }
    }

    public static void main(String[] args) {

        int graph[][] = {
            {0, 4, 0, 0, 0, 0},
            {4, 0, 8, 0, 0, 0},
            {0, 8, 0, 7, 2, 4},
            {0, 0, 7, 0, 9, 14},
            {0, 0, 2, 9, 0, 10},
            {0, 0, 4, 14, 10, 0}
        };

        DijkstraAlgorithm obj = new DijkstraAlgorithm();
        obj.dijkstra(graph, 0); // Source vertex = 0
    }
}
```

Output:

```
Vertex  Distance from Source
0       0
1       4
2       12
3       19
4       14
5       16
```

Possible Viva Questions:

1. What problem does Dijkstra's algorithm solve in computer networks?
2. Why do we use a large number like 9999 in the adjacency matrix?
3. Which real-world routing protocol is based on Dijkstra's algorithm?
4. Can Dijkstra's algorithm work with negative weights? Why?
5. How is Dijkstra's different from Bellman-Ford algorithm?

Result:

EXP – 04	Take an example subnet of hosts and obtain a broadcast tree for the subnet.
Date:	

Aim:
To construct a **broadcast tree** for a given subnet and minimize redundant transmissions.

Description:

A **broadcast tree** connects all nodes in a subnet with **minimum cost** without creating loops.
It ensures every node receives the broadcast efficiently.
We can use **Prim's Algorithm (Minimum Spanning Tree)** to generate the broadcast tree.

Program: Broadcast Tree (Java Program)

```
import java.util.Scanner;
public class BroadcastTree {
   public static void main(String[] args) {
      Scanner sc = new Scanner(System.in);
      final int INF = 9999;
      System.out.print("Enter number of nodes: ");
      int n = sc.nextInt();
      int[][] graph = new int[n][n];
      boolean[] visited = new boolean[n];
      System.out.println("Enter adjacency matrix (9999 for no edge):");
      for (int i = 0; i < n; i++)
         for (int j = 0; j < n; j++)
            graph[i][j] = sc.nextInt();

      visited[0] = true;
      int edges = 1, cost = 0;
      System.out.println("\nBroadcast Tree Edges:");
      while (edges < n) {
         int min = INF, u = -1, v = -1;
         for (int i = 0; i < n; i++) {
            if (visited[i]) {
               for (int j = 0; j < n; j++) {
                  if (!visited[j] && graph[i][j] < min) {
                     min = graph[i][j];
                     u = i;
                     v = j;
                  }
               }
            }
         }
         System.out.println("Edge " + edges + ": (" + u + " -> " + v + ") cost = " + min);
         cost += min;
         visited[v] = true;
         edges++;
      }
      System.out.println("\nTotal Broadcast Tree Cost = " + cost);
      sc.close();
   }
```

}

Output:

Adjacency Matrix

1 2	3 4	6 5
2 3	7 7	1 1
4 3	5 4	8 9

Enter number of nodes: 3
Enter adjacency matrix (9999 for no edge):
12
34
65
23
77
11
43
54
89

Possible Viva Questions:

1. What is a broadcast tree in a network?
2. Why do we use a Minimum Spanning Tree (MST) to build a broadcast tree?
3. Which algorithms are commonly used to compute MST?
4. How does Prim's Algorithm differ from Kruskal's Algorithm?
5. How does a broadcast tree prevent loops in a network?

Result:

EXP – 05	Implement distance vector routing algorithm for obtaining routing tables at each node.
Date:	

Aim:
To implement the **Distance Vector Routing Algorithm** and compute the routing tables for each node in a network.

Description:
The program reads the **cost matrix** of the network (999 indicates no direct link).
It computes the shortest distances between all pairs of nodes using the **Distance Vector Routing algorithm** (Bellman-Ford/Floyd-Warshall approach).
Finally, it displays the **routing table** for each node, showing the minimum cost to every destination node.

Program: Distance Vector Routing Algorithm (Java Program)

```
import java.util.Scanner;

public class DistanceVector {
   public static void main(String[] args) {
      Scanner sc = new Scanner(System.in);
      final int INF = 9999;

      System.out.print("Enter number of nodes: ");
      int n = sc.nextInt();
      int[][] cost = new int[n][n];
      int[][] dist = new int[n][n];

      System.out.println("Enter cost adjacency matrix (9999 for no direct path):");
      for (int i = 0; i < n; i++)
         for (int j = 0; j < n; j++) {
            cost[i][j] = sc.nextInt();
            dist[i][j] = cost[i][j];
          }
      // Distance Vector Algorithm
      for (int k = 0; k < n; k++)
         for (int i = 0; i < n; i++)
            for (int j = 0; j < n; j++)
               if (dist[i][k] + dist[k][j] < dist[i][j])
                  dist[i][j] = dist[i][k] + dist[k][j];

      // Display routing table
      System.out.println("\nRouting Table (Shortest Distances):");
      for (int i = 0; i < n; i++) {
         System.out.print("From Node " + i + ": ");
         for (int j = 0; j < n; j++)
            System.out.print(dist[i][j] + " ");
         System.out.println();
      }

      sc.close();
```

```
    }
}
```

Output:

Enter number of nodes: 3
Enter cost adjacency matrix (9999 for no direct path): Adjacency Matrix

1	0	3
2	0	2
3	0	1

1
0
3
2
0
2
3
0
1

Possible Viva Questions:

1. What is the basic principle of Distance Vector Routing?
2. How does a router update its routing table?
3. What is the role of neighbouring nodes in this algorithm?
4. What is the difference between Distance Vector and Link State Routing?
5. Can Distance Vector Routing handle negative edge weights?

Result:

EXP – 06	Implement data encryption and data decryption.
Date:	

Aim:
To implement a simple **encryption and decryption program** for secure data transmission.

Description:

Encryption converts plain text into unreadable form (cipher text) to prevent unauthorized access.
Decryption converts cipher text back to plain text.
Caesar Cipher is one of the simplest encryption techniques: each character is shifted by a fixed number of positions.
This experiment demonstrates basic cryptography at the application level.

Program: Encryption and Decryption (Java Program)

```
import java.util.Scanner;

public class EncryptionDecryption {
   public static void main(String[] args) {
      Scanner sc = new Scanner(System.in);

      System.out.print("Enter text: ");
      String text = sc.nextLine();

      System.out.print("Enter shift key: ");
      int key = sc.nextInt();

      String encrypted = "", decrypted = "";

      for (char ch : text.toCharArray()) {
         if (Character.isLetter(ch)) {
            char base = Character.isUpperCase(ch) ? 'A' : 'a';
            encrypted += (char)((ch - base + key) % 26 + base);
            decrypted += (char)((ch - base - key + 26) % 26 + base);
         } else {
            encrypted += ch;
            decrypted += ch;
         }
      }

      System.out.println("Encrypted Text: " + encrypted);
      System.out.println("Decrypted Text: " + text);

      sc.close();
   }
}
```

Output:

Enter text: HELLO
Enter shift key: 3

Possible Viva Questions:

1. What is encryption and why is it important?
2. How does the Caesar Cipher work?
3. Can Caesar Cipher be easily broken? Why?
4. What is the difference between symmetric and asymmetric encryption?
5. Give examples of real-world applications of encryption.

Result:

EXP – 07	Write a program for congestion control using Leaky bucket algorithm.
Date:	

Aim:
To implement the **Leaky Bucket algorithm** in Java to control network congestion.

Description:
The **Leaky Bucket** algorithm controls data flow in networks.
Packets arrive at different rates but leave the bucket at a **constant rate**, preventing congestion.
Excess packets are either **queued** or **dropped** when the bucket is full.

Program: Leaky Bucket Algorithm (Java Program)

```
import java.util.Scanner;

public class LeakyBucket {
   public static void main(String[] args) {
      Scanner sc = new Scanner(System.in);

      System.out.print("Enter bucket capacity: ");
      int capacity = sc.nextInt();

      System.out.print("Enter number of packets: ");
      int n = sc.nextInt();

      int[] packets = new int[n];
      System.out.println("Enter packet sizes:");
      for (int i = 0; i < n; i++) packets[i] = sc.nextInt();

      int bucket = 0, rate = 5; // fixed output rate

      for (int i = 0; i < n; i++) {
         bucket += packets[i];
         if (bucket > capacity) {
            System.out.println("Packet " + (i+1) + " dropped");
            bucket = capacity;
         } else {
            System.out.println("Packet " + (i+1) + " accepted");
         }
         System.out.println("Bucket size after sending: " + Math.max(bucket - rate, 0));
         bucket = Math.max(bucket - rate, 0);
      }

      sc.close();
   }
}
```

Output:

Enter bucket capacity: 10
Enter number of packets: 5
Enter packet sizes:
4
6
3
5
2

Possible Viva Questions:

1. What is network congestion?
2. How does the Leaky Bucket algorithm prevent congestion?
3. What happens if incoming packets exceed bucket capacity?
4. What is the difference between Leaky Bucket and Token Bucket algorithms?
5. Why is a fixed output rate important in Leaky Bucket?

Result:

EXP – 08	Write a program for frame sorting technique used in buffers.
Date:	

Aim:
To implement a simple **frame sorting technique** in Java to organize frames in a buffer for proper sequence.

Description:
In data communication, frames may arrive **out of order** due to network delays.
Frame sorting ensures the **correct sequence** before processing.
Sorting can be done using simple **ascending order** based on frame numbers.

Program: Frame Sorting (Java Program)

```
import java.util.Scanner;
import java.util.Arrays;

public class FrameSorting {
   public static void main(String[] args) {
      Scanner sc = new Scanner(System.in);

      System.out.print("Enter number of frames: ");
      int n = sc.nextInt();

      int[] frames = new int[n];
      System.out.println("Enter frame numbers:");
      for (int i = 0; i < n; i++) frames[i] = sc.nextInt();

      Arrays.sort(frames);

      System.out.println("Frames in correct sequence:");
      for (int f : frames) System.out.print(f + " ");

      sc.close();
   }
}
```

Output:

```
Enter number of frames: 5
Enter frame numbers:
4
1
3
5
2
```

Possible Viva Questions:

1. Why do frames need to be sorted in a buffer?
2. What problems occur if frames are processed out of order?
3. Can this technique handle missing frames? How?
4. Which sorting algorithm is used in this program?
5. How is frame sorting related to flow control in networks?

Result:

EXP – 09	**Programs using Wireshark:** **i. Starting Wireshark** **ii. Packet Capture Using Wireshark** **iii. Viewing Captured Traffic** **iv. Analysis and Statistics & Filters**
Date:	

Aim:

To start **Wireshark** & capture, view, and analyse network packets using Wireshark, and to understand network protocols, packet structure, and traffic flow in detail.

Description:

Wireshark is a **graphical network protocol analyser** widely used in computer networking for **capturing and analysing data packets** in real-time. It provides detailed insights into **how data travels in networks** , enabling the study of network communication, protocol behaviour, and packet structures.

Key Features of Wireshark:

1. **Real-Time Packet Capture:**
   - Captures packets from network interfaces such as Ethernet, Wi-Fi, or VPN connections.
   - Displays live traffic including local and external communications.
2. **Multi-Protocol Support:**
   - Can decode hundreds of protocols including **TCP, UDP, HTTP, FTP, ICMP, DNS, ARP, and DHCP**.
   - Helps understand layered communication in computer networks.
3. **Packet Inspection:**
   - Provides detailed **protocol header information** and **payload content**.
   - Displays information at **different layers** : Physical, Data Link, Network, Transport, and Application.
4. **Filters and Search:**
   - **Display filters:** Narrow down packet list to specific protocols or IP addresses.
   - **Capture filters:** Limit the capture to only certain traffic types.
5. **Statistics and Analysis:**
   - **Protocol Hierarchy:** Displays the percentage of packets per protocol.
   - **IO Graphs:** Visualizes traffic flow over time.
   - **Conversations and Endpoints:** Analyse communication between specific devices.
6. **Packet Saving and Export:**
   - Captured packets can be saved in .pcap format for further analysis or use in external tools like **Pcap4J in Java**.
7. **Educational and Diagnostic Tool:**
   - Helps understand **network handshakes, packet flow, congestion, and errors** .
   - Useful for troubleshooting, performance analysis, and protocol study.

Wireshark Interface:

The interface is divided into **three main sections** :

Packet List Pane: Displays all captured packets with basic information like No., Time, Source, Destination, Protocol, Length, and Info.

Packet Details Pane: Shows detailed header fields for each protocol layer within the selected packet.

Packet Bytes Pane: Displays raw hexadecimal and ASCII data for each packet.

Procedure / Sections:

i. Starting Wireshark:

Open Wireshark application.
Select the **network interface** to monitor.
Click the **Start Capture (green shark fin)** button.
Packets begin appearing in real-time.

Observations:

Packets are continuously updated in the **Packet List Pane**.
Capture can be stopped at any time and saved for further analysis.

ii. Packet Capture Using Wireshark:

Captured packets appear in the **Packet List Table**, each representing a unit of network data.
Packets include headers and payload information.

Sample Packet List Table:

No.	Time	Source	Destination	Protocol	Length	Info
1	0.0001	192.168.1.2	192.168.1.1	TCP	60	SYN
2	0.0005	192.168.1.1	192.168.1.2	TCP	60	SYN, ACK
3	0.0010	192.168.1.2	192.168.1.1	TCP	54	ACK
4	0.0020	192.168.1.2	192.168.1.1	HTTP	512	GET /index.html
5	0.0025	192.168.1.1	192.168.1.2	HTTP	1024	200 OK

Observations:

Source and destination addresses indicate the sender and receiver.
Protocol column shows whether the packet uses TCP, UDP, HTTP, etc.
Length column indicates the size of the packet in bytes.
Info column provides a brief summary of packet purpose, e.g., handshake or HTTP request.

iii. Viewing Captured Traffic (Packet Details and Bytes):

Packet Details Table (Example for Packet 2):

Layer	Field	Value
Ethernet II	Destination	00:1A:2B:3C:4D:5E
	Source	11:22:33:44:55:66
	Type	IPv4
Internet Protocol	Version	4
	Source IP	192.168.1.1
	Destination IP	192.168.1.2
Transmission Control Protocol	Source Port	443
	Destination Port	53245
	Flags	SYN, ACK

Packet Bytes Pane (Hexadecimal & ASCII Representation):

```
0000  00 1A 2B 3C 4D 5E 11 22 33 44 55 66 08 00 45 00
0010  00 34 00 00 40 00 40 06 00 00 C0 A8 01 01 C0 A8
0020  01 02 01 BB D0 69 00 00 00 00 70 02 FA F0 00 00
0030  00 00 02 04 05 B4 01 03 03 08
```

Observations:

Ethernet, IP, and TCP layers are clearly visible in structured form.
Hexadecimal data represents the raw packet information transmitted over the network.
ASCII representation allows inspection of readable payload content.

iv. Analysis, Statistics & Filters:

Display Filters:

tcp – Shows only TCP packets.
http – Shows only HTTP packets.
ip.addr==192.168.1.1 – Shows packets sent or received by a specific IP.

Protocol Hierarchy Table:

Protocol	Packets	Percentage
TCP	25	50%
UDP	15	30%
ICMP	10	20%

IO Graphs:

Graphical representation of packets over time.
Peaks indicate high traffic periods, while flat lines indicate idle network periods.

Conversations and Endpoints Analysis:

Shows communication between two devices.
Useful for identifying traffic patterns or heavy communication nodes.

Observations:

Protocol distribution helps analyze the type of traffic in the network.
IO graphs help in identifying traffic congestion and network performance.
Conversations highlight main communicators and endpoints.

Possible Viva Questions:

1. What is Wireshark and what is its purpose?
2. What are the three main panes in the Wireshark interface?
3. How can a specific protocol like TCP or HTTP be isolated?
4. What information does the Packet Details Pane provide?
5. What is the importance of Protocol Hierarchy and IO Graphs in analysis?

Result:

EXP – 10	How to run Nmap scan.
Date:	

Aim:
To perform network scanning and detect operating systems of hosts using **Nmap**, and analyze open ports and network services.

Description:

Nmap (Network Mapper) is a powerful open-source tool used for **network discovery and security auditing** . It allows scanning of hosts, identification of open ports, running services, and detection of the operating system.

Key Features of Nmap:

1. **Host Discovery:** Identifies live hosts on a network.
2. **Port Scanning:** Detects open, closed, and filtered ports on a host.
3. **Service Version Detection:** Identifies software running on detected ports.
4. **Operating System Detection:** Estimates the OS of remote hosts.
5. **Flexible Output:** Provides results in text, XML, or HTML formats.

Applications:

Network auditing
Security testing
Network mapping
Detecting unauthorized devices

Nmap Installation on Windows:

Step 1: Download Nmap

1. Visit the official website: https://nmap.org/download.html
2. Download the latest **Windows installer** (e.g., nmap-7.xx-setup.exe).

Step 2: Install Nmap

1. Run the downloaded installer.
2. Follow the installation wizard:
   - Accept license agreement.
   - Choose installation location (default is recommended).
   - Select optional components like **Zenmap (GUI)** if needed.
3. Complete the installation and click **Finish**.

Step 3: Set Environment Variables

1. Press Windows + S → type **Environment Variables** → select **Edit the system environment variables** .
2. In the **System Properties** window, click **Environment Variables** .
3. Under **System variables** , find the **Path** variable and click **Edit**.
4. Click **New** and add the **Nmap installation directory path** , e.g.:
   C:\Program Files (x86)\Nmap
5. Click **OK** to close all windows.

Step 4: Verify Installation

Open **Command Prompt** (cmd) and type:

nmap –version

Output:

Program: Performing Nmap Scan (Java Program)

```
import java.io.BufferedReader;
import java.io.InputStreamReader;
import java.util.Arrays;

public class NmapScan {
   public static void main(String[] args) {
      try {
         String target = "192.168.1.1"; // Replace with target IP
         ProcessBuilder pb = new ProcessBuilder("nmap", "-O", target);
         pb.redirectErrorStream(true); // Merge stderr with stdout
         Process process = pb.start();

         BufferedReader reader = new BufferedReader(new
InputStreamReader(process.getInputStream()));
         String line;

         System.out.println("---- Nmap Scan Output ----");
         while ((line = reader.readLine()) != null) {
            System.out.println(line);
         }

         int exitCode = process.waitFor();
         System.out.println("---- Scan Completed with exit code: " + exitCode + " ----");

      } catch (Exception e) {
         e.printStackTrace();
      }
   }
}
```

Output:

Possible Viva Questions:

1. What is Nmap and what is it used for?
2. How are open ports detected using Nmap?
3. What does the -O option do?
4. How can environment variables be set for Nmap in Windows?
5. How can Nmap results help in network security auditing?

Result:

EXP – 11	Operating System Detection using Nmap.
Date:	

Aim:
To detect operating systems of multiple hosts on a network using **Nmap**, analyze open ports, running services, and identify network characteristics.

Description:

Nmap (Network Mapper) is an open-source network scanning tool used for **network discovery, security auditing, and OS detection** . Advanced OS detection uses **TCP/IP stack fingerprinting** to identify the operating system running on a host.

Key Concepts:

1. **TCP/IP Fingerprinting:**
   - Nmap sends specially crafted packets to a host and analyzes the responses.
   - Differences in responses help identify the OS type, version, and sometimes kernel.
2. **Port and Service Scanning:**
   - Detects open, closed, or filtered ports on the target.
   - Determines running services and their versions.
3. **Network Topology Analysis:**
   - Identifies network distance (number of hops).
   - Detects firewalls and packet filtering devices.
4. **Advanced OS Detection Options:**
   - -O → Enables OS detection.
   - --osscan-guess → Provides a probable OS if exact match is uncertain.
   - -v → Verbose output for detailed packet and host info.

Applications:

Network auditing and monitoring
Security testing for vulnerabilities
Identifying unauthorized devices

Program: Nmap OS Detection (Java Program)

```
import java.io.BufferedReader;
import java.io.InputStreamReader;

public class NmapOSDetection {
   public static void main(String[] args) {
      try {
         // Array of target IP addresses
         String[] targets = {"192.168.1.1", "192.168.1.2"};

         for (String target : targets) {
            System.out.println("---- Scanning Host: " + target + " ----");

            // Using ProcessBuilder to run Nmap safely
            ProcessBuilder pb = new ProcessBuilder("nmap", "-O", "-v", target);
            pb.redirectErrorStream(true); // Combine stdout and stderr
            Process process = pb.start();
```

```
            // Read output of the Nmap process
            BufferedReader reader = new BufferedReader(new
InputStreamReader(process.getInputStream()));
            String line;
            while ((line = reader.readLine()) != null) {
                System.out.println(line);
            }

            // Wait for process to complete
            process.waitFor();
            System.out.println("---- Scan Completed for " + target + " ----\n");
        }
    } catch (Exception e) {
        e.printStackTrace();
    }
  }
}
```

Output:

Possible Viva Questions:

1. What is the purpose of OS detection in Nmap?
2. How does Nmap identify the operating system of a host?
3. What is TCP/IP fingerprinting?
4. How do you run an Nmap scan for a single host?
5. How can Nmap detect the number of hops to a host?

Result:

EXP – 12	**Do the following using NS2 Simulator:** **i. NS2 Simulator – Introduction** **ii. Simulate to Find the Number of Packets Dropped** **iii. Simulate to Find the Number of Packets Dropped by TCP/UDP** **iv. Simulate to Find the Number of Packets Dropped due to Congestion** **v. Simulate to Compare Data Rate & Throughput** **vi. Simulate to Plot Congestion for Different Source/Destination** **vii. Simulate to Determine the Performance with respect to transmission of Packets**
Date:	

Aim:

To simulate and analyse network performance using **NS2 Simulator** including:

i. Introduction to NS2 Simulator
ii. Packet drops
iii. TCP/UDP flow behaviour
iv. Congestion effects
v. Data rate and throughput comparison
vi. Congestion plotting for different source/destination pairs
vii. Performance analysis with respect to packet transmission

Description:

Network Simulator 2 (NS2):

NS2 is a discrete event simulator used for modeling and simulating network protocols and traffic. It supports both **wired and wireless networks** . NS2 allows detailed analysis of network performance using trace files, and provides **NAM (Network Animator)** to visualize packet flows.

Key Features:

1. **Discrete Event Simulation:** All events such as sending, receiving, or dropping packets are processed in a sequential manner, allowing accurate modeling.
2. **Protocol Support:** TCP, UDP, routing protocols, and application layer protocols like FTP and CBR are supported.
3. **Traffic Analysis:** Trace files record packet-level events for detailed analysis.
4. **Visualization (NAM):** NAM provides an animation of network topology and packet flow.
5. **Performance Metrics:** NS2 allows calculating throughput, packet delivery ratio, packet drop ratio, and congestion patterns.

Key Concepts:

Packet Drop: Occurs due to congestion or link errors.
TCP Flow: Provides reliable transmission using retransmissions.
UDP Flow: Connectionless, faster but may lose packets under congestion.
Congestion: Occurs when multiple flows exceed link capacity.
Throughput: Data successfully delivered per unit time.
Packet Delivery Ratio: Fraction of sent packets successfully delivered.

Applications:

Performance evaluation of protocols
Congestion analysis
Network planning
Research and academic studies

Procedure / Steps with Detailed Explanations and Sample Outputs:

i. NS2 Simulator – Introduction

Explanation:

Before simulating traffic, NS2 must be installed and configured. NS2 scripts are written in **TCL** language to define network topology, nodes, links, agents, and applications. NAM visualizes the simulation.

Procedure:

1. Install NS2 from http://www.isi.edu/nsnam/ns/.
2. Open a terminal and check installation using following commands:
   ns
   nam
3. Define nodes and links in a TCL script.

Output (Text-Based NAM Visualization):

Node0 ---> Node1: TCP Flow
Packets move along the link
Link bandwidth: 1 Mbps
Delay: 10 ms

Observation: The network topology is correctly visualized. Packets flow from source to destination as per simulation.

ii. Simulate to Find the Number of Packets Dropped

Explanation:

Packet drops occur due to network congestion or delay. Simulating a single TCP flow allows observing basic packet drop behavior.

Procedure:

Create two nodes connected via a 1 Mbps link.
Attach TCP agent to Node0, TCPSink to Node1.
Start FTP application to generate traffic.

Output Table:

Metric	Value
Packets Sent	1000
Packets Received	950
Packets Dropped	50

Observation: 50 packets dropped due to network delay and minor congestion on the link.

iii. Simulate to Find Packets Dropped by TCP/UDP

Explanation:

TCP retransmits lost packets; UDP does not. This section shows the difference in reliability between TCP and UDP flows.

Procedure:

Add TCP flow (Node0→Node1) and UDP flow (Node2→Node3).
Send traffic simultaneously and monitor trace file.

Output Table:

Flow	Sent	Received	Dropped
TCP	1000	950	50
UDP	1200	1100	100

Observation: TCP has fewer packet drops due to retransmission, while UDP drops more packets under same conditions.

iv. Simulate to Find Packets Dropped due to Congestion

Explanation:
Multiple flows through a single bottleneck cause congestion, resulting in increased packet drops.

Procedure:
Create multiple TCP flows through a bottleneck link (1 Mbps).
Monitor trace file or NAM for dropped packets.

Output Table:

Flow	Sent	Received	Dropped
TCP1	800	750	50
TCP2	700	660	40

Observation: Congestion occurs at bottleneck link; flows experience packet loss.

v. Simulate to Compare Data Rate & Throughput

Explanation:
Data rate affects throughput. By changing link bandwidth, we can compare performance.

Procedure:
Flow1 link bandwidth = 500 Kbps, Flow2 link bandwidth = 2 Mbps.
Monitor received packets and calculate throughput.

Output Table:

Flow	Data Rate	Throughput
Flow1	500 Kbps	480 Kbps
Flow2	2 Mbps	1.95 Mbps

Observation: Throughput slightly lower than data rate due to overhead. Higher bandwidth improves performance.

vi. Simulate to Plot Congestion for Different Source/Destination

Explanation:
Traffic flows through shared links result in congestion. Packet drop patterns help plot

congestion.

Procedure:

Multiple nodes connected via shared links.
Monitor packet drops for each flow.

Output Table:

Source → Destination	Sent	Received	Dropped
Node0 → Node1	1000	950	50
Node2 → Node3	800	760	40
Node4 → Node5	900	850	50

Observation: Congestion higher on links shared by multiple flows; packet drops indicate stress points.

vii. Simulate to Determine Performance w.r.t Transmission of Packets

Explanation:

This section calculates **packet delivery ratio, packet drop ratio, and throughput** to evaluate network efficiency.

Output Table:

Metric	Value
Packets Sent	1000
Packets Received	950
Packets Dropped	50
Packet Delivery Ratio (%)	95
Packet Drop Ratio (%)	5
Throughput (Mbps)	0.95

Observation: TCP provides reliable transmission; throughput is slightly less than link capacity.

Possible Viva Questions:

1. What is NS2 and its applications?
2. How do TCP and UDP differ in NS2 simulations?
3. How do you measure packet drops using NS2 trace files?
4. What is the purpose of NAM?
5. How is throughput calculated from NS2 output?

Result: